

A1 Agent Engine Platform Guide

A comprehensive guide to the A1 Agent Engine Admin Console and Agent Studio, designed for platform operators, domain architects, and system administrators.

Table of Contents

1. [Overview](#overview)
2. [Admin Console](#admin-console)
3. [Agent Studio](#agent-studio)
4. [Core Concepts](#core-concepts)
5. [Common Workflows](#common-workflows)
6. [Best Practices](#best-practices)

Overview

The A1 Agent Engine is a **Temporal-based durable agent orchestration platform** that enables enterprises to build, deploy, and manage autonomous agents at scale.

Two-User Model

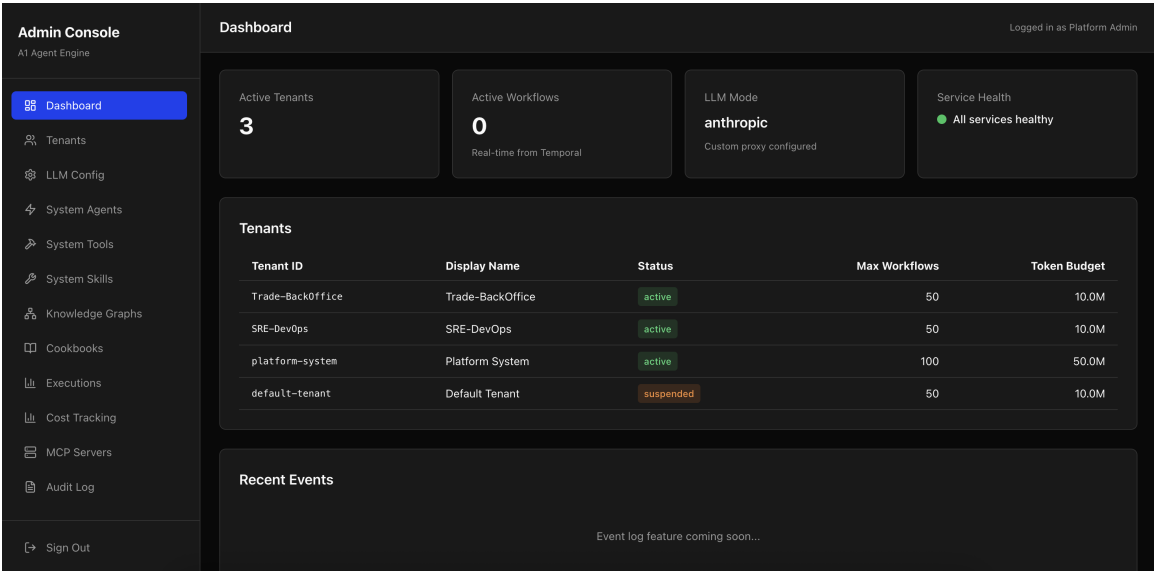
The platform serves two distinct user personas:

1. **Platform Operators** (Admin Console)
 - Manage infrastructure and global platform configuration
 - Configure LLM providers and model access
 - Monitor system health and usage
 - Manage MCP servers and external integrations
 - View audit logs and cost tracking
2. **Domain Architects** (Agent Studio)
 - Design and compose agents from skills
 - Create and manage domain-specific knowledge graphs
 - Import and customize cookbooks (templates)
 - Build and test agents in sandbox environment
 - Monitor agent execution and approvals

Admin Console

The Admin Console is the platform management interface. Access at **localhost:3001** (development) or your production domain.

1. Dashboard



1. Dashboard Interface

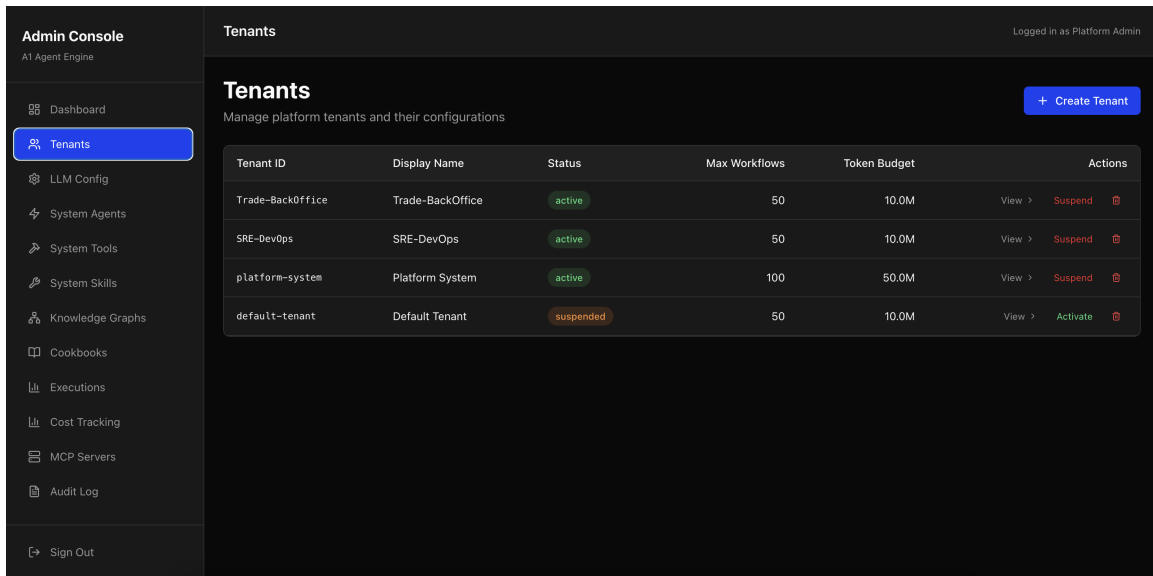
Purpose: Real-time overview of platform health and activity

Key Metrics:

- **Active Tenants:** Number of tenant workspaces
- **Active Workflows:** Real-time Temporal workflow count
- **LLM Mode:** Current LLM provider (Mock, Anthropic, OpenAI)
- **Service Health:** Status of all microservices
- **Tenant Management:** View all registered tenants with workflow quotas and token budgets

Screenshot: Platform Dashboard showing 5 active tenants, real-time workflow monitor, and service health status.

2. Tenant Management



2. Tenant Management Interface

Purpose: Multi-tenant workspace administration

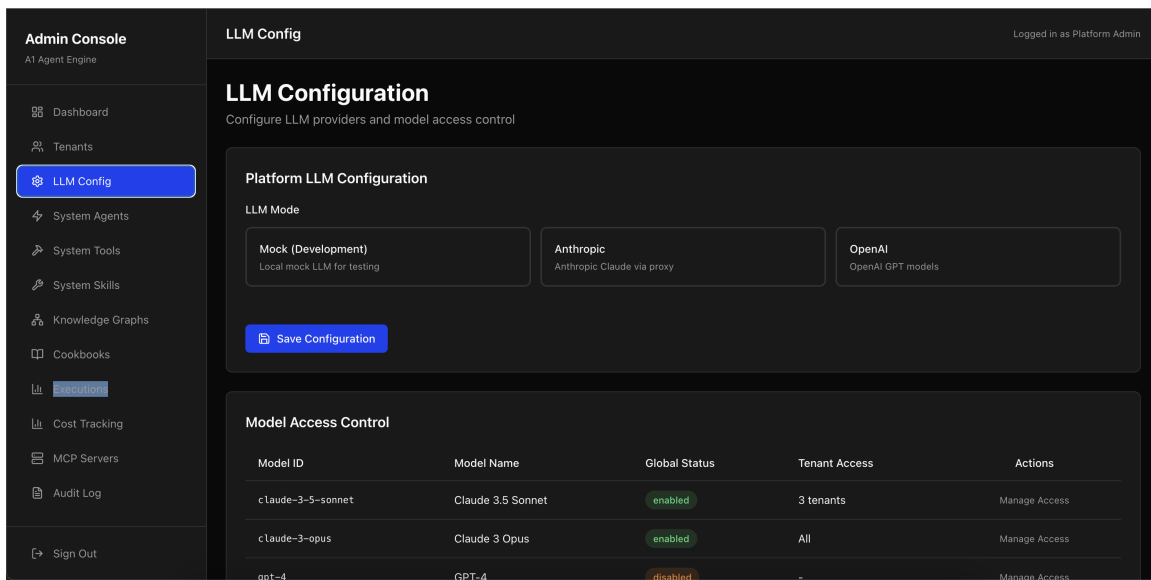
Features:

- Create and configure isolated tenant environments
- Set workflow execution quotas per tenant
- Manage token budgets and rate limits
- Monitor tenant-specific usage and costs
- Enable/disable tenants as needed

Tenant Isolation:

- PostgreSQL Row-Level Security (RLS) enforces data isolation
- Each agent, skill, and tool belongs to a specific tenant
- Cross-tenant access is technically impossible

3. LLM Configuration



3. LLM Configuration Interface

Purpose: Configure LLM providers and manage model access

Configuration Options:

Platform LLM Mode:

- **Mock (Development):** Deterministic responses for testing
- **Anthropic:** Claude models via Anthropic API
- **OpenAI:** GPT models via OpenAI API

Model Access Control:

Configure which models are available:

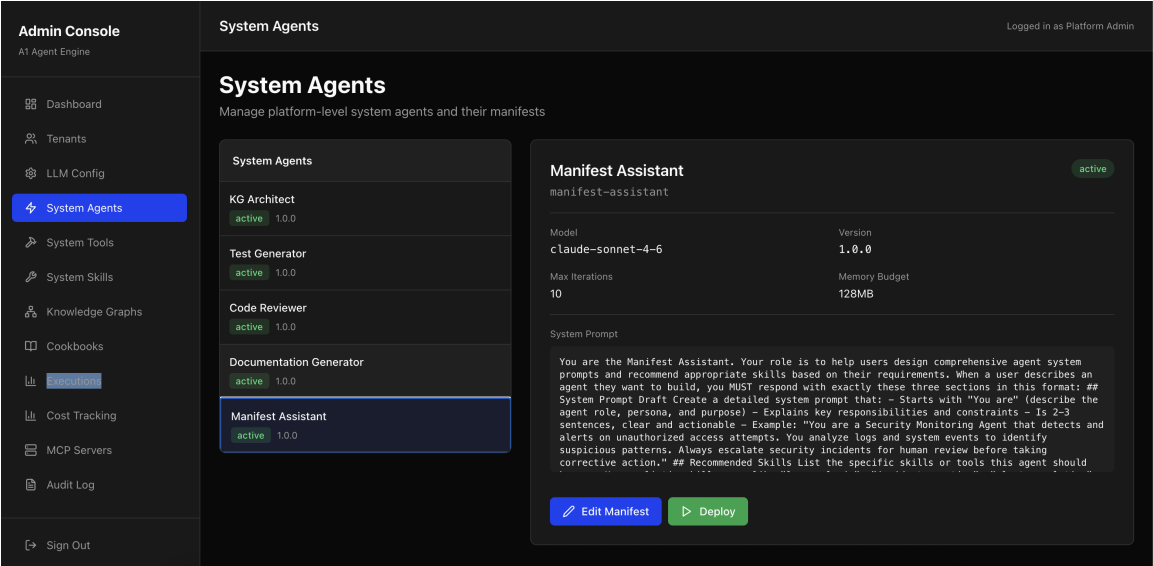
- **claude-3-5-sonnet** - Recommended for balanced performance
- **claude-3-opus** - Most capable, for complex reasoning
- **gpt-4** - OpenAI alternative

Per-Model Configuration:

- Enable/disable models globally or per-tenant
- Set API key routing
- Configure fallback models
- Monitor token usage and costs

Screenshot: LLM Configuration page showing Mock, Anthropic, and OpenAI provider options with Model Access Control table (Claude 3.5 Sonnet, Claude 3 Opus, GPT-4).

4. System Agents



4. System Agents Interface

Purpose: Pre-built system agents for platform operations

Available System Agents:

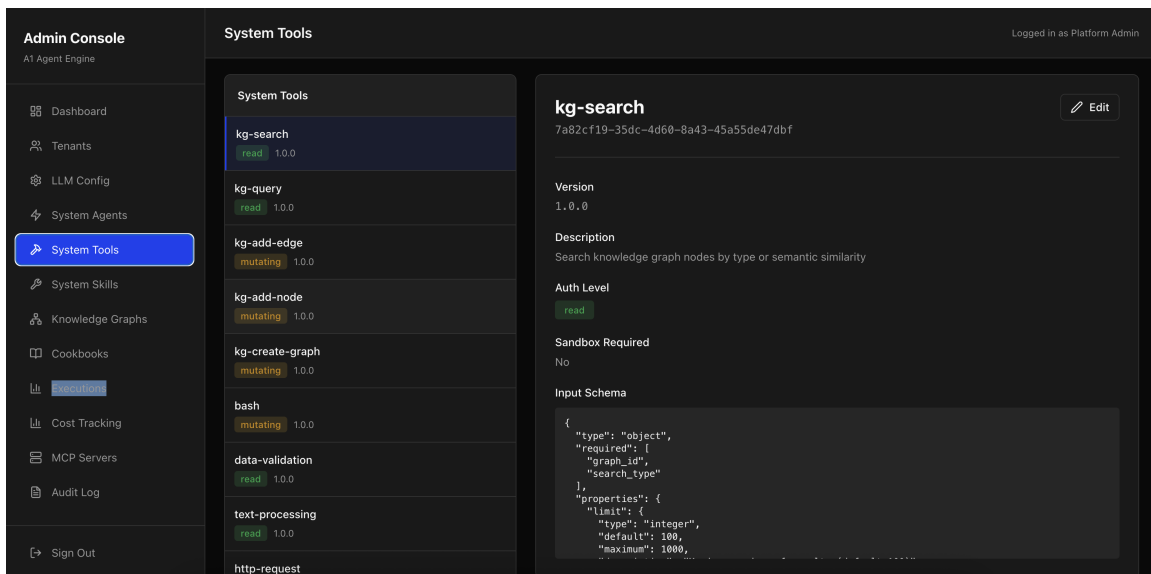
Agent	Version	Use Case
Manifest Assistant	1.0.0	Help with agent manifest generation
Test Generator	1.0.0	Generate unit tests for code changes
Code Reviewer	1.0.0	Peer code review automation
Deployment Checker	1.0.0	Validate deployment configurations

Features:

- View agent manifests
- Enable/disable per tenant
- Monitor execution logs
- Manage agent versions

Screenshot: System Agents page listing pre-built agents (Manifest Assistant, Test Generator, Code Reviewer) with manifest viewer showing detailed agent configuration.

5. System Tools



5. System Tools Interface

Purpose: Platform-level tools available to all agents

Tool Categories:

Infrastructure Tools:

- **bash** - Shell command execution (sandbox-controlled)
- **deployment-checker** - Kubernetes deployment validation
- **log-analyzer** - Log aggregation and analysis

Knowledge Graph Tools:

- **kg-create-graph** - Create knowledge graphs
- **kg-add-node** - Add nodes to KGs
- **kg-add-edge** - Create relationships
- **kg-query** - Query KGs
- **kg-search** - Full-text search over KGs
- **kg-semantic-search** - NLP-powered semantic search

Data Tools:

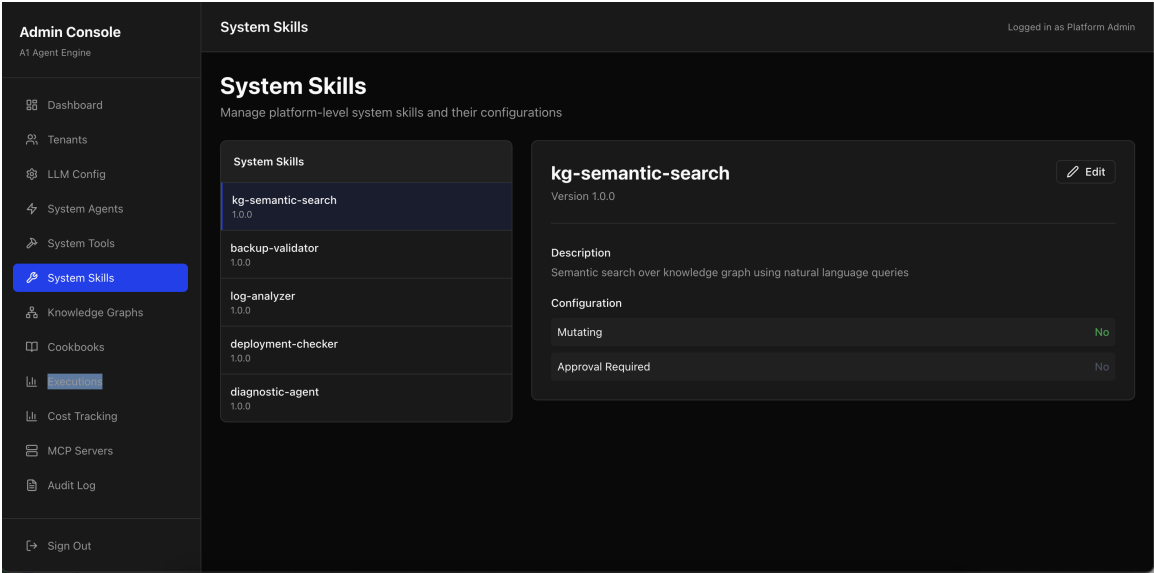
- **http-request** - HTTP API calls
- **code-executor** - Python/JavaScript execution (sandboxed)

Tool Configuration:

- View tool documentation
- Track tool usage across tenants

- Monitor execution metrics

6. System Skills



6. System Skills Interface

Purpose: Pre-built domain skills for reuse across agents

Available System Skills:

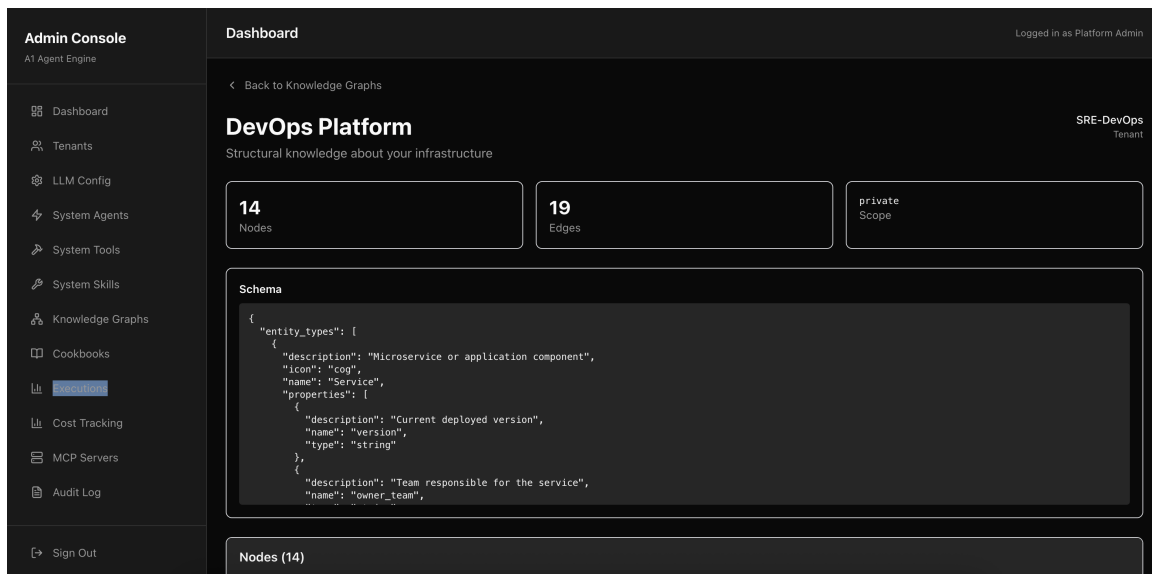
Skill	Mutating	Requires Approval
kg-semantic-search	No	No
backup-validator	No	No
log-analyzer	No	No
deployment-checker	Yes	Yes
diagnostic-agent	Yes	Yes

Features:

- Browse skill catalog
- View skill documentation
- Configure approval requirements (HITL)
- Monitor skill usage

Screenshot: System Skills page showing kg-semantic-search skill details (description, configuration, mutating flag, approval required flag).

7. Knowledge Graphs



7. Knowledge Graphs Interface

Purpose: Store and manage domain ontologies and semantic relationships

Knowledge Graph Features:

Structure:

- **Nodes:** Entities (services, teams, deployments)
- **Edges:** Relationships (owns, depends-on, monitors)
- **Schema:** Type definitions and constraints
- **Embedding:** Auto-embedding of node data for semantic search

Example: DevOps Platform KG

Shows infrastructure topology:

- 14 nodes (Kubernetes clusters, services, teams)
- 19 edges (ownership, dependency relationships)
- Schema: Service definitions, team hierarchies, deployment models

Visualization:

- Interactive node-link diagram
- Filter by node type
- Search across the graph

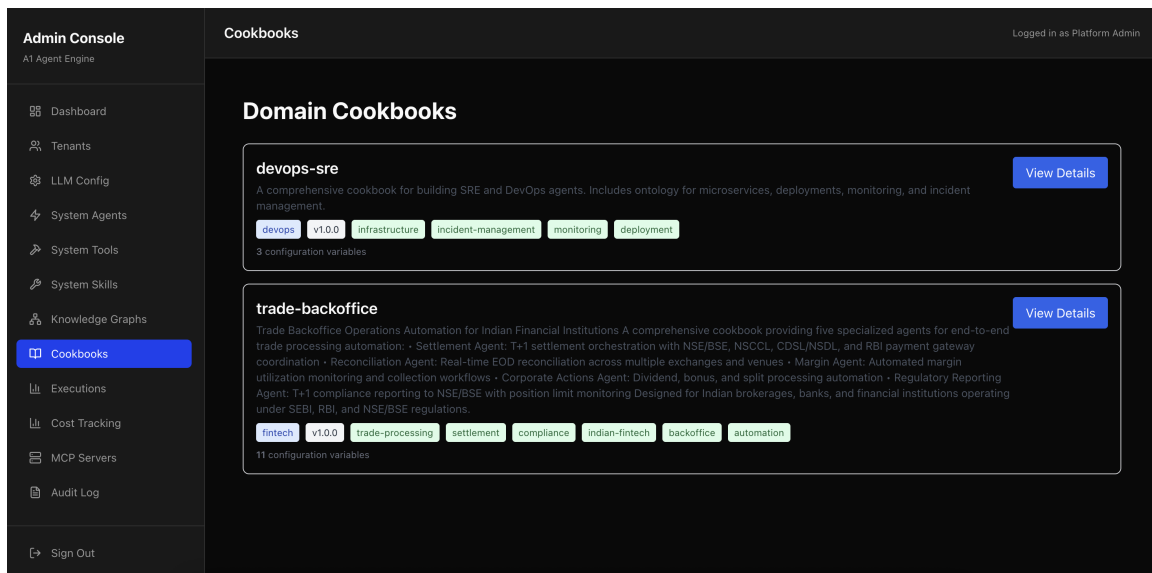
- View node attributes in sidebar

Use Cases:

- Infrastructure relationship mapping
- Service dependency graphs
- Team hierarchy and ownership
- Feature/capability trees
- Policy compliance mappings

Screenshot: Knowledge Graph visualization of DevOps Platform showing 14 nodes (services, teams) and 19 relationships with interactive filtering.

8. Cookbooks



8. Cookbooks Interface

Purpose: Domain-specific agent templates for rapid deployment

Cookbook Anatomy:

Overview:

- Description and use case
- Domain tags (e.g., SRE, DevOps, Security)
- Version and maintenance status
- Import action for using in Agent Studio

Components:

- **Variables:** Parameterized configuration
- **org_name** - Organization identifier
- **env_names** - Comma-separated environments
- **alert_channel** - Notification destination
- **Agents:** Pre-built agent definitions
- Count and descriptions
- **Knowledge Graphs:** Domain ontologies
- Included KGs and their structures
- **MCP Recommendations:** Suggested MCP servers
- Server configurations
- API integrations

Example: DevOps-SRE Cookbook

Description: "Comprehensive cookbook for building SRE and DevOps agents. Includes ontology for microservices, deployments, monitoring, and incident management."

Variables:

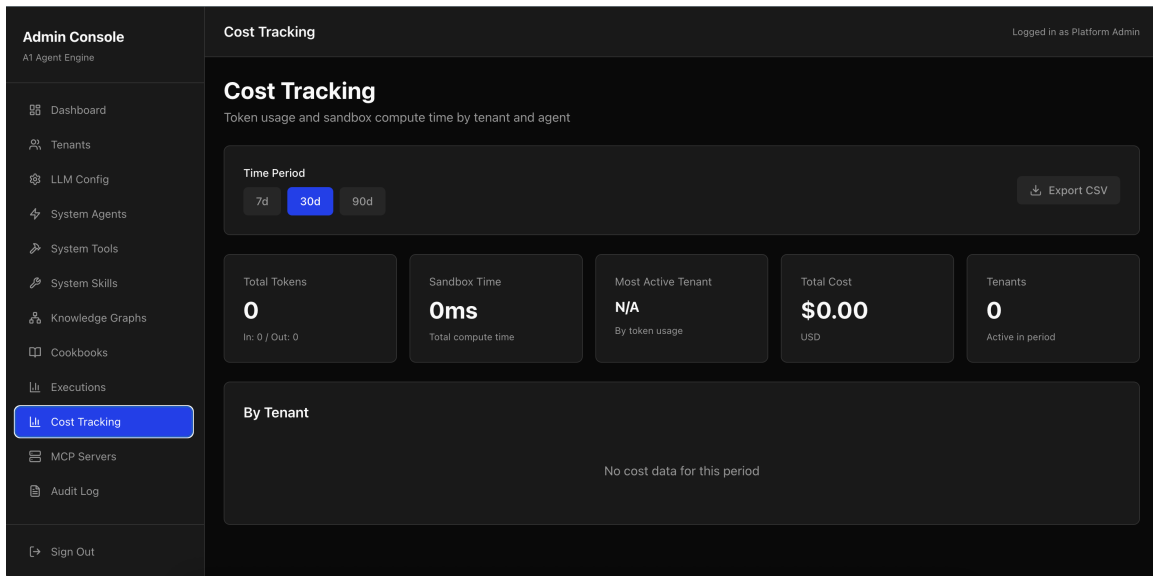
- **org_name** - Your organization name
- **env_names** - dev,staging,prod
- **alert_channel** - slack

Artifacts:

- 2 Agents (e.g., deployment-validator, incident-responder)
- 1 Knowledge Graph (DevOps Platform ontology)
- 8 MCP Recommendations (monitoring, incident management)

Screenshot: Cookbook detail showing devops-sre with variables table, domain tags (devops, sre, infrastructure, incident-management, monitoring, deployment), and artifacts summary.

9. Cost Tracking



9. Cost Tracking Interface

Purpose: Monitor platform usage and billing

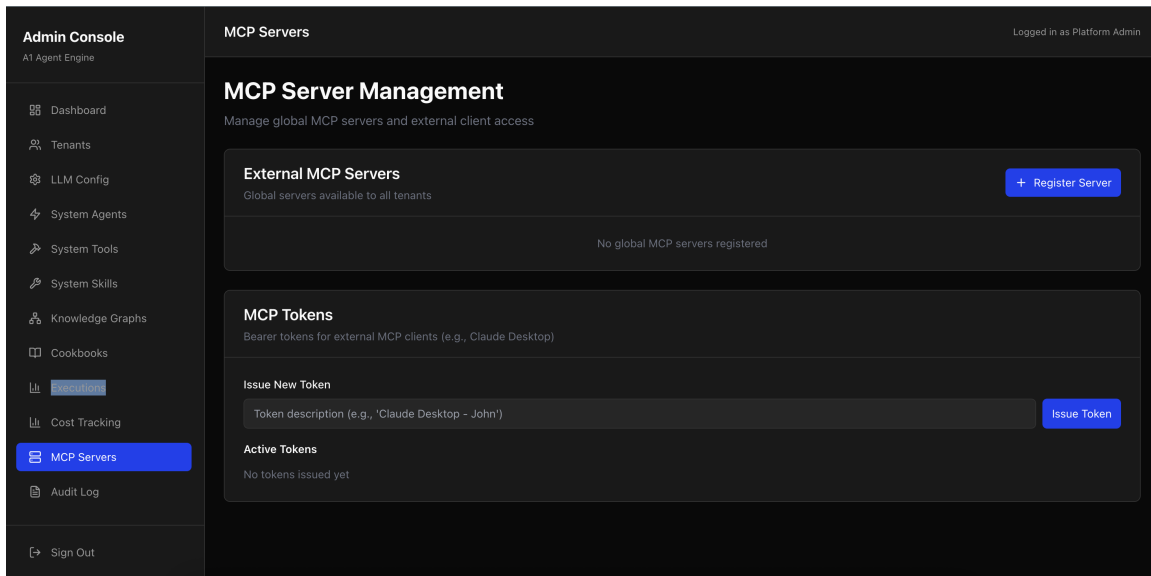
Metrics Tracked:

- Token consumption per tenant
- API calls and transaction counts
- Model-specific costs (Claude vs GPT)
- Workflow execution minutes
- Storage usage (KGs, agent state)

Reporting:

- Real-time usage dashboard
- Cost attribution by tenant
- Monthly billing summaries
- Custom cost allocation rules

10. MCP Servers



10. MCP Servers Interface

Purpose: Manage Model Context Protocol integrations

External MCP Servers:

- Register global MCP servers available to all tenants
- Configure authentication (API keys, OAuth)
- Monitor server health and connectivity

MCP Tokens:

- Issue tokens for external MCP clients (e.g., Claude Desktop)
- Manage token lifecycle and permissions
- Revoke tokens as needed

Benefits:

- Integrate external AI models (Claude Desktop, other providers)
- Extend platform with custom tools via MCP protocol
- Manage multi-agent ecosystems

Screenshot: MCP Server Management showing External MCP Servers section, MCP Tokens (no tokens issued yet), and Issue New Token button with description field.

11. Audit Log

Purpose: Complete audit trail of platform operations

Events Logged:

- Tenant and user management
- LLM configuration changes
- Workflow executions
- Tool invocations and approvals
- MCP server registrations
- Cost and usage changes

Audit Features:

- Filter by date, tenant, user, action
- Export audit logs
- Search by keywords
- Compliance reporting

Agent Studio

The Agent Studio is the workspace for domain architects to design and test agents. Access at **localhost:3000** (development) or your production domain.

1. Tools

Purpose: Explore and understand available tools

Tool Information:

- Tool name and version
- Description and use case
- Parameter definitions
- Usage examples
- Availability status

Tool Categories:

- Infrastructure tools (bash, deployment-checker)
- Data tools (http-request, code-executor)
- Knowledge graph tools (kg-search, kg-semantic-search)
- Platform tools (skill-dispatcher)

2. Skills

Purpose: Reusable agent capabilities built from tools

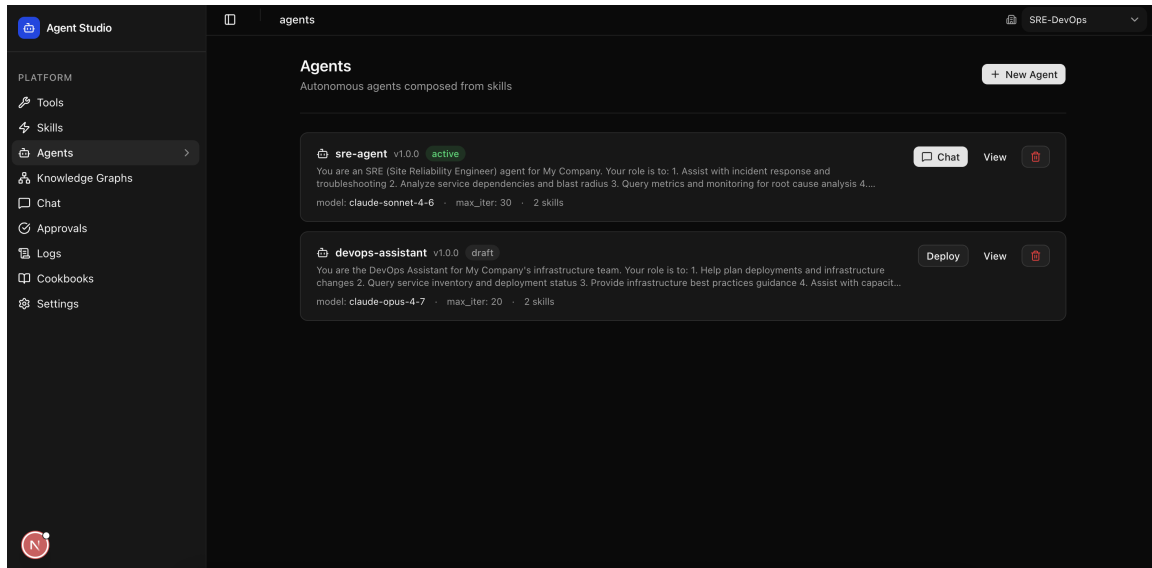
Skill Structure:

- Name and description
- Tools used (dependencies)
- Configuration parameters
- Approval requirements

Skill Reuse:

- Add skills to agents
- Combine multiple skills
- Skill versioning and updates
- Approval workflows

3. Agents



3. Agents Interface

Purpose: Create and manage autonomous agents

Agent Definition:

- Name and description

- Model selection (Claude, GPT)
- Max iterations
- Skills composition
- System prompt customization
- Tools access (read-only or full)

Agent Lifecycle:

- **Draft:** In development
- **Active:** Ready for use
- **Deprecated:** No longer recommended
- **Archived:** Legacy agents

Agent Capabilities:

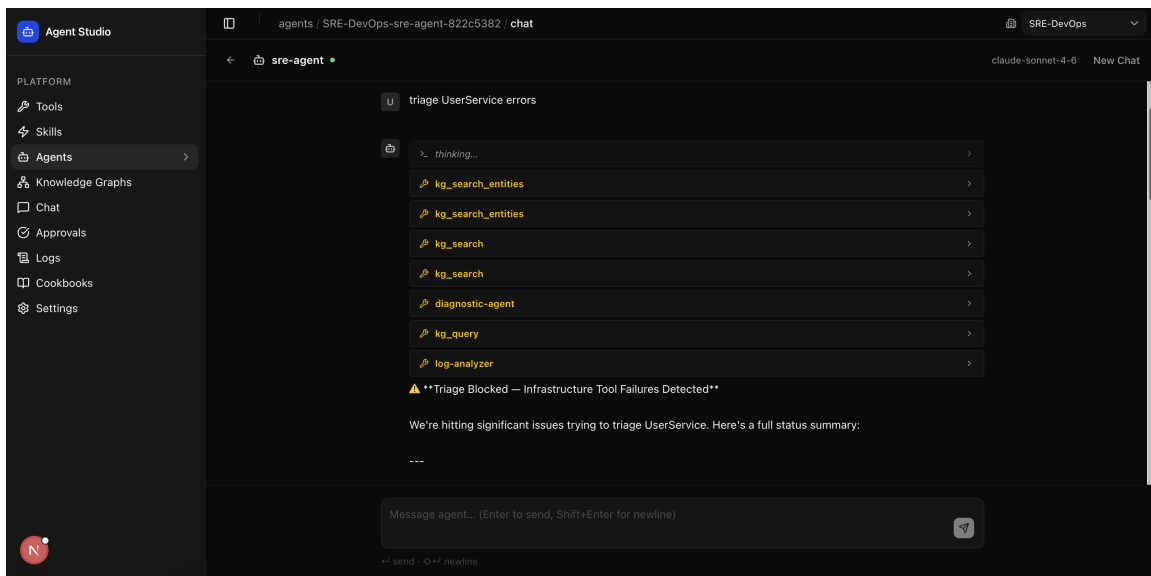
- Orchestrate skills and tools
- Maintain conversation context
- Execute workflows
- Track state and decisions

Example: Test Chat Agent

- **Model:** mock-model (for testing)
- **Max Iterations:** 5
- **Description:** Helpful assistant for testing chat functionality
- **Status:** Active
- **Actions:** Chat, View, Delete

Screenshot: Agents page showing "Test Chat Agent v1.0.0 (active)" with description, model configuration, and action buttons.

4. Chat



4. Chat Interface

Purpose: Test agents in real-time conversation

Chat Features:

- Send messages to agents
- View agent reasoning and tool calls
- Monitor token usage
- Export conversation history
- Track execution logs

Agent Response Process:

1. Agent receives user message
2. Plans approach using available tools
3. Executes tools (bash, API calls, KG searches)
4. Iterates until task complete or max iterations reached
5. Returns final response to user

Example Chat Session:

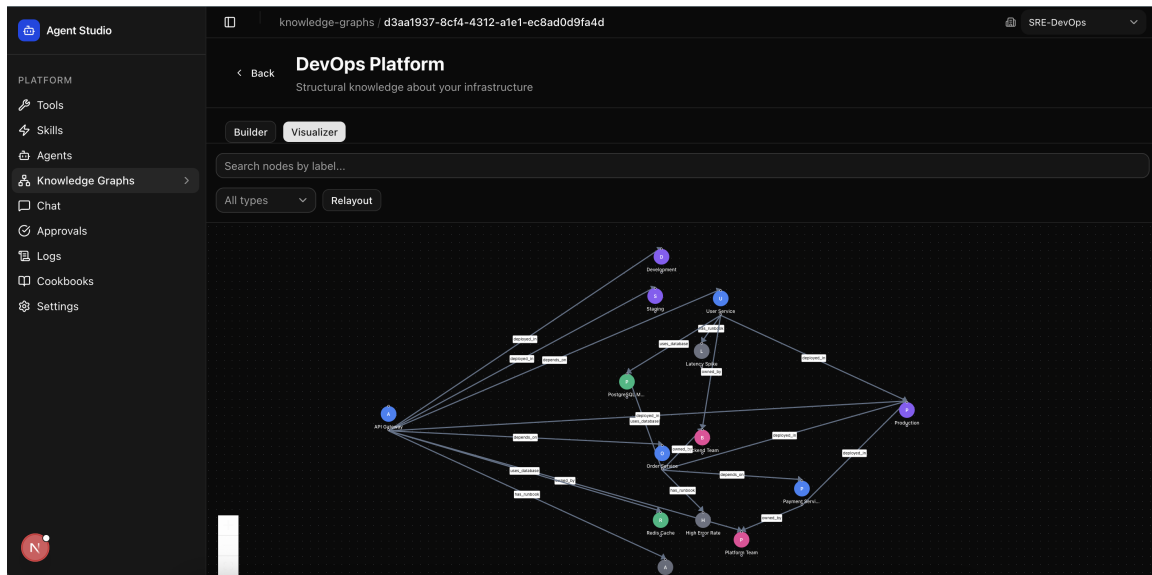
Agent responses show:

- Tool invocations (e.g., **kg_search_entities**, **kg_search_entities**, **kg_search**, **kg_search**, **diagnostic_agent**)
- Error handling ("Image blocked — Infrastructure Tool Failures Detected")
- Partial results and fallbacks

- Conversation context preservation

Screenshot: Chat interface showing agent executing multiple tool calls with "Image blocked — Infrastructure Tool Failures Detected" warning and list of attempted tool invocations (kg_search_entities, kg_search, diagnostic_agent, etc.).

5. Knowledge Graphs



5. Knowledge Graphs Interface

Purpose: Browse and visualize domain knowledge

KG Interaction:

- Visual node-link diagram
- Filter by node type
- Search for specific nodes
- View node properties
- Explore relationships

KG Features:

- Semantic search over graph
- Schema validation
- Embedding visualization
- Version history

Use Cases:

- Understand domain structure
- Validate agent knowledge
- Identify knowledge gaps
- Plan knowledge enrichment

Screenshot: Knowledge Graph visualization showing interactive graph view of DevOps Platform with color-coded nodes, relationship edges, and builder/visualizer toolbar.

6. Approvals

Purpose: Human-in-the-loop approval for mutating operations

Approval Workflow:

- Agent requests approval for sensitive actions
- Operator reviews context and validates action
- Approve or reject with reasoning
- Audit trail maintained

Approval Types:

- Infrastructure changes (deployment, configuration)
- Data modifications (delete, update)
- High-risk operations (bash execution, API calls)

Approval Interface:

- View pending approvals
- Review action context
- Approve or reject with notes
- View approval history

7. Logs

Purpose: Debug and monitor agent execution

Log Information:

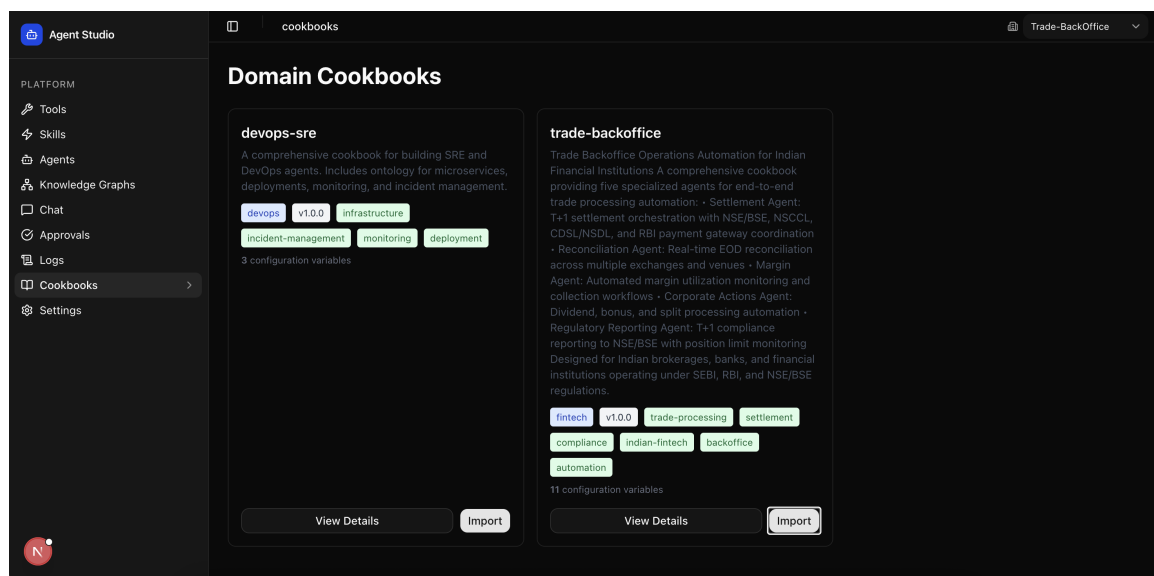
- Execution timestamp
- Agent name and version

- Tool calls made
- Results and errors
- Token usage
- Execution duration

Log Analysis:

- Filter by agent, time range
- Search error messages
- Export logs
- Compare execution patterns

8. Cookbooks



8. Cookbooks Interface

Purpose: Access and customize domain templates

Cookbook Workflow:

1. **Browse** available cookbooks
2. **Review** template structure (agents, KGs, variables)
3. **Import** cookbook into your tenant
4. **Customize** variables for your environment
5. **Deploy** agents and KGs

6. **Extend** with custom modifications

Cookbook Customization:

- Override variables
- Extend agents with new skills
- Enhance knowledge graphs
- Add custom tools

Example: Importing DevOps-SRE Cookbook

Variables to configure:

- **org_name** → "My Company"
- **env_names** → "dev,staging,prod"
- **alert_channel** → "#incidents"

Result:

- 2 agents deployed (deployment-validator, incident-responder)
- DevOps Platform KG created (14 nodes, 19 edges)
- 8 MCP recommendations configured

Screenshot: Cookbook detail page showing devops-sre cookbook with domain tags, variables, and "Import Cookbook" action button.

9. **Settings**

Purpose: Configure workspace preferences

Settings:

- Default model selection
- API keys and credentials
- Notification preferences
- Export settings
- Tenant information

Core Concepts

1. **Agents**

Definition: Autonomous entities that reason about problems, invoke tools, and execute tasks.

Properties:

- **Model:** LLM backbone (Claude, GPT)
- **Skills:** Reusable capabilities
- **Tools:** Direct access to system functions
- **State:** Memory of past interactions
- **Constraints:** Max iterations, token limits

Execution Model (Temporal):

- All agent runs are durable workflows
- Survives failures and restarts
- Audit trail of execution
- Horizontal scaling via worker pools

2. Skills

Definition: Reusable agent capabilities composed from tools.

Characteristics:

- Encapsulate domain logic
- Abstract tool complexity
- Versioned and discoverable
- Approval-aware (HITL)
- Tenant-scoped

Skill Composition:

Skill = Tool + Configuration + Approval Rules

Example:

kg-semantic-search skill uses kg-semantic-search tool
backup-validator skill uses bash + monitoring tools

3. Tools

Definition: Atomic functions agents invoke (bash, API, KG search, etc.).

Tool Categories:

Category	Tools	Safety Model
----------	-------	--------------

Infrastructure	bash, deployment-checker	Sandboxed, approval required
Knowledge	kg-search, kg-semantic-search	Read-only by default
Data	http-request, code-executor	Approval required
Platform	skill-dispatcher	Internal routing

Mutating vs. Read-Only:

- **Mutating:** Modify state (require approval)
- **Read-Only:** Query operations (unrestricted)

4. Knowledge Graphs

Definition: Semantic graphs storing domain ontologies and relationships.

Structure:

- **Nodes:** Typed entities (Service, Team, Infrastructure)
- **Edges:** Relationships with labels (owns, depends-on)
- **Attributes:** Node properties (name, description, status)
- **Schema:** Type system and constraints

Capabilities:

- Full-text search
- Semantic similarity search
- Relationship traversal
- Schema validation
- Version tracking

Use Cases:

- Represent service architectures
- Model team hierarchies
- Store policy compliance mappings
- Document infrastructure dependencies

5. Cookbooks

Definition: Domain-specific templates for rapid agent composition.

Anatomy:

- **Description:** Use case and domain
- **Variables:** Parameterized configuration

- **Agents:** Pre-built agent templates
- **KGs:** Domain ontologies
- **Recommendations:** Suggested tools/integrations

Benefits:

- Accelerate agent development
- Enforce domain best practices
- Ensure consistency across agents
- Enable knowledge sharing

Example Domains:

- DevOps & SRE
- Security & Compliance
- Infrastructure Management
- Incident Response

6. Multi-Tenancy

Isolation Model:

- PostgreSQL Row-Level Security (RLS) at DB layer
- Tenant-scoped API keys and tokens
- Separate Temporal task queues (per tenant)
- Database connection pooling per tenant

Data Isolation:

- Agents belong to exactly one tenant
- Skills scoped to tenant (can be "platform" for all)
- Knowledge graphs are tenant-scoped
- Cost tracking by tenant

Implications:

- Secure cross-customer deployments
- Independent scaling per tenant
- Separate billing and quotas
- Compliance boundaries

7. HITL (Human-In-The-Loop) Approval

Concept: Critical operations require human authorization before execution.

Approval Types:

- Mutating tools (bash, deployment changes)
- High-risk workflows (data deletion, API calls)
- Policy exceptions
- Manual interventions

Workflow:

1. Agent plans action requiring approval
2. Sends approval request with context
3. Human reviewer examines and decides
4. Approval or rejection with reasoning
5. Agent proceeds or halts
6. Audit trail recorded

Benefits:

- Safety for production operations
- Compliance with governance
- Transparency in decision-making
- Reduces accidents and errors

Common Workflows

Workflow 1: Set Up a New Tenant

Steps:

1. **Admin Console → Tenants**
 - Create new tenant
 - Set workflow quota (e.g., 1000/day)
 - Set token budget (e.g., 100K/day)
2. **Admin Console → LLM Config**
 - Enable models for this tenant
 - (Optional) Set model-specific quotas

3. **Agent Studio** → **Settings**

- Configure default model
- Set API keys if needed
- Review tenant isolation

4. **Verify Isolation**

- Confirm agents are tenant-scoped
- Check knowledge graphs belong to tenant
- Validate cost tracking by tenant

Workflow 2: Deploy a Domain Agent

Steps:

1. **Agent Studio** → **Cookbooks**

- Browse available templates
- Select relevant domain cookbook

2. **Review Cookbook**

- Understand agents, KGs, tools
- Check variables and customization points

3. **Import Cookbook**

- Provide variable values (org_name, env_names, etc.)
- Confirm resources to be created

4. **Customize Agents**

- Extend agent with additional skills
- Adjust model selection if needed
- Test in chat interface

5. **Deploy**

- Mark agent as "active"
- Configure approval workflows
- Enable for end-users

Workflow 3: Create a Custom Agent

Steps:

1. **Agent Studio** → **Agents** → **New Agent**

- Name and description
- Select model (Claude, GPT)
- Set max iterations

2. **Compose from Skills**

- Select relevant skills
- Configure skill parameters
- Add approval requirements

3. **Test in Chat**

- Send test messages
- Monitor tool invocations
- Verify responses

4. **Review Logs**

- Check execution traces
- Identify tool failures
- Optimize skill selection

5. **Deploy**

- Set status to "active"
- Configure access controls
- Document usage

Workflow 4: Troubleshoot Agent Failures

Steps:

1. **Agent Studio** → **Logs**

- Find failing execution
- Review tool calls and outputs
- Identify error point

2. **Check Tool Availability**

- Admin Console → System Tools
- Verify tool is enabled
- Check approval requirements

3. Review Skills

- Verify skill uses correct tool
- Check skill configuration
- Test skill in isolation

4. Examine Knowledge Graph

- Agent Studio → Knowledge Graphs
- Search for related entities
- Validate relationships

5. Update Agent

- Adjust skill selection
- Refine system prompt
- Increase max iterations if needed

6. Test Again

- Re-run in chat interface
- Verify fix works
- Export logs for documentation

Workflow 5: Approve Agent Operation

Steps:

1. Agent Studio → Approvals

- View pending approval requests
- Read agent reasoning and proposed action

2. Review Context

- Understand why agent is taking action
- Check execution logs
- Validate against policy

3. Make Decision

- Approve if reasonable
- Reject with reasoning if not
- Add comments for audit trail

4. Follow Up

- Monitor execution after approval
- Review logs and results
- Provide feedback to agent developers

Best Practices

1. Agent Design

Do:

- Compose agents from well-tested skills
- Use descriptive system prompts
- Limit max iterations to prevent runaway
- Enable HITL approval for risky operations
- Test extensively before deploying

Don't:

- Give agents unlimited tool access
- Mix too many unrelated skills
- Use overly verbose system prompts
- Deploy to production without testing
- Ignore approval requirements

2. Skill Development

Do:

- Make skills single-purpose and focused
- Document expected inputs and outputs
- Version skills explicitly
- Test with multiple models
- Collect usage metrics

Don't:

- Create monolithic "do everything" skills
- Assume specific tool behavior
- Introduce breaking changes without version bump

- Ignore error handling
- Skip documentation

3. Knowledge Graph Management

Do:

- Keep schemas consistent and well-documented
- Update graphs regularly with current state
- Validate graph integrity
- Use semantic relationships meaningfully
- Enable agents to discover context

Don't:

- Treat KGs as one-time snapshots
- Allow schema drift
- Create disconnected subgraphs
- Overload nodes with unrelated attributes
- Forget to embed node data

4. Cookbook Creation

Do:

- Include comprehensive documentation
- Parameterize configuration with variables
- Provide example values
- Test import process
- Package complete solutions

Don't:

- Hard-code environment-specific values
- Ship partial or incomplete templates
- Forget to include all required skills/tools
- Use overly complex variable structures
- Skip validation

5. Approval Workflows

Do:

- Be conservative with approval requirements
- Require approval for all mutating operations
- Provide clear, actionable approval requests
- Maintain comprehensive audit trails
- Review approval logs regularly

Don't:

- Approve actions without understanding context
- Create infinite approval loops
- Ignore approval rejections
- Allow unapproved mutating operations
- Forget to document approval decisions

6. Monitoring & Observability

Do:

- Track agent execution metrics
- Monitor tool invocation patterns
- Alert on failure rates
- Export logs for analysis
- Review cost trends regularly

Don't:

- Ignore failing agents
- Assume everything is working
- Skim monitoring dashboards
- Let logs grow unbounded
- Miss early signs of issues

7. Security & Compliance

Do:

- Enforce tenant isolation
- Rotate MCP tokens regularly
- Audit access patterns

- Enable approval for sensitive ops
- Keep audit logs retained

Don't:

- Bypass approval workflows
- Mix secrets in logs
- Grant excessive permissions
- Trust unvalidated inputs
- Delete audit logs prematurely

8. Documentation

Do:

- Document agent purpose and limitations
- Explain skill usage and dependencies
- Keep KG schemas documented
- Provide cookbook usage examples
- Record approval decisions

Don't:

- Assume people understand implicit behavior
- Let documentation rot
- Mix documentation with code
- Skip examples
- Forget to update docs with changes

Support & Resources

Documentation

- **[README.md](./README.md)** — Project overview and quick start
- **[architecture.md](./architecture.md)** — System design and data flow
- **[requirements.md](./requirements.md)** — Feature spec and SLOs

Temporal Workflows

- [Temporal Documentation](https://docs.temporal.io) — Durable workflow concepts
- [Temporal SDK Reference](https://docs.temporal.io/sdks) — Multi-language SDKs

Troubleshooting

Common Issues:

Issue	Solution
Agent fails with "Tool not found"	Check tool is enabled in System Tools
"Approval required" errors	Review HITL approval settings in Skills
Knowledge graph search returns no results	Verify KG is imported and embedded
Cookbook import fails	Check variables are provided correctly
Chat timeout or max iterations exceeded	Increase max_iter or simplify task

Getting Help

1. **Agent Studio Logs** — Detailed execution traces
2. **Admin Console Audit Log** — System-wide event history
3. **Documentation** — Architecture and configuration guides
4. **Support Team** — Escalation for platform issues

Appendix: API Endpoints

Admin API (Port 8089)

Endpoint	Method	Purpose
/health`	GET	Health check
/tenants`	GET/POST	Manage tenants
/tenants/{id}`	GET/PUT/DELETE	Tenant details
/agents`	GET	List agents
/agents/{id}`	GET	Agent details
/workflows/{id}`	GET	Workflow status
/approvals`	GET/POST	Manage approvals

Public API (Port 8080)

Endpoint	Method	Purpose
<code>/chat`</code>	POST	Send chat message to agent
<code>/agents/{id}`</code>	GET	Get agent definition
<code>/workflows`</code>	POST	Start new workflow
<code>/workflows/{id}`</code>	GET	Check workflow status

For complete API documentation, see [\[docs/api.md\]\(./docs/api.md\)](#).

Last Updated: May 15, 2026

Version: 1.0.0

Maintained By: Arun Ray

For feedback or corrections, please open an issue in the project repository.